# EXPERIMENT 4

**Aim :** To understand Continuous Integration, install and configure Jenkins with Maven/Ant/Gradle to setup a build Job

## Theory:

Continuous Integration (CI) is a modern software development practice in which developers frequently merge their code changes into a shared repository. Each integration is automatically verified through build and testing processes. The main objective of CI is to identify errors early, improve software quality, and ensure that the application remains stable throughout development.

In a CI workflow, developers use a Version Control System (VCS) such as Git or SVN to manage source code. Whenever a change is committed, the CI system automatically triggers a build process. This build process compiles the source code into executable form using tools like Maven, Ant, or Gradle. Automation in building helps reduce human errors and ensures consistency across all builds.

Testing is a crucial component of CI. Automated tests, including unit tests, integration tests, and functional tests, are executed during the build process to verify that new changes do not break existing functionality. This ensures reliability and helps maintain code quality.

A CI server, such as Jenkins, plays a central role in managing the entire process. Jenkins continuously monitors the repository for any updates, executes builds, runs test cases, and provides feedback to developers. It also supports plugins that allow integration with various build tools like Maven, Ant, and Gradle.

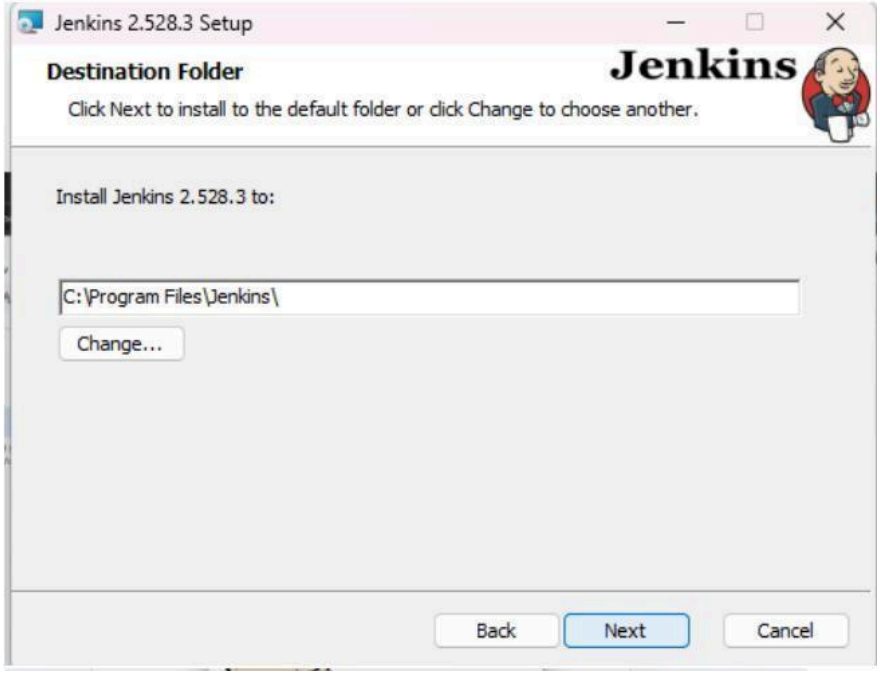
To implement CI using Jenkins, the first step is installing Jenkins and

starting its service. After installation, the initial configuration is completed through a web-based setup wizard where essential plugins are installed. Next, a new job is created in Jenkins by selecting a project type such as a Freestyle project or Pipeline.

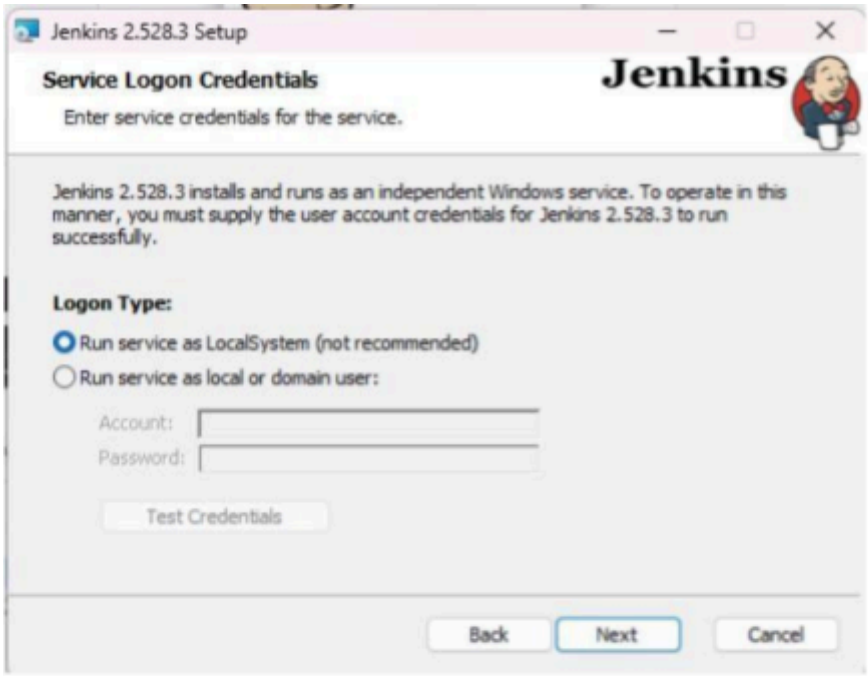
The job is then configured by linking it to a source code repository and defining build triggers. The appropriate build tool is selected—Maven, Ant, or Gradle—and its configuration is specified. For example, in Maven, goals like *clean* and *install* are defined. Once the configuration is saved, the build can be executed manually or automatically. The console output helps in identifying any issues during the process.

Continuous Integration provides several advantages in software development. It enables early detection of defects, ensures consistent and repeatable builds, improves collaboration among developers, and provides quick feedback on code changes. Overall, CI helps in delivering reliable and high-quality software efficiently.

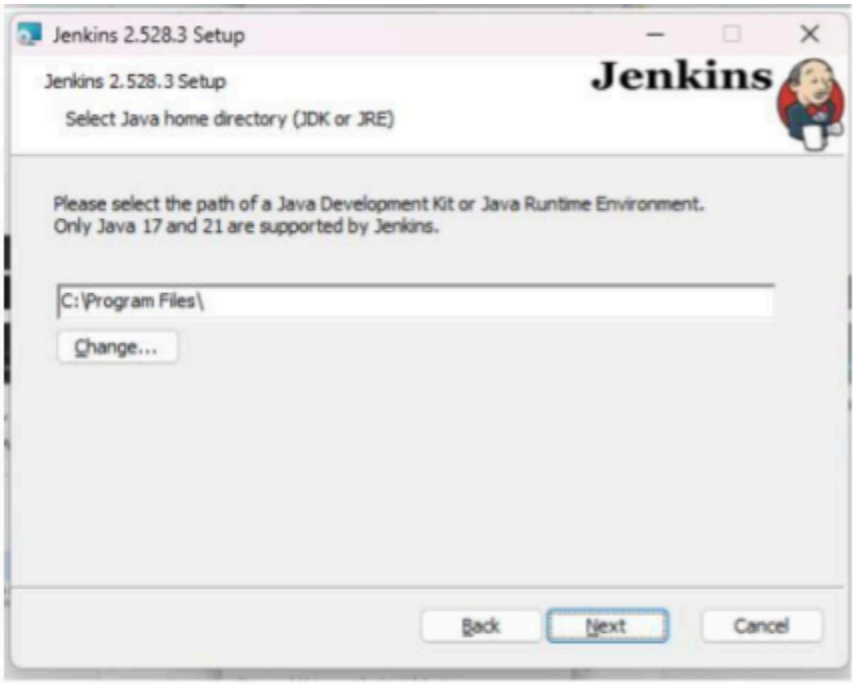
**Name : Kunal Motwani** **Batch : T21** **Roll No : 72**

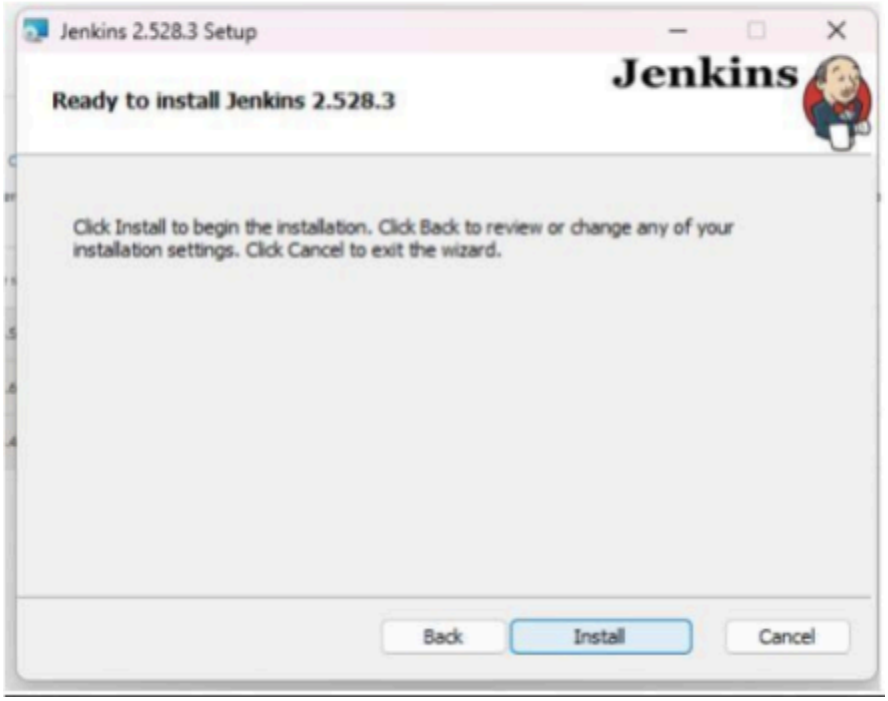


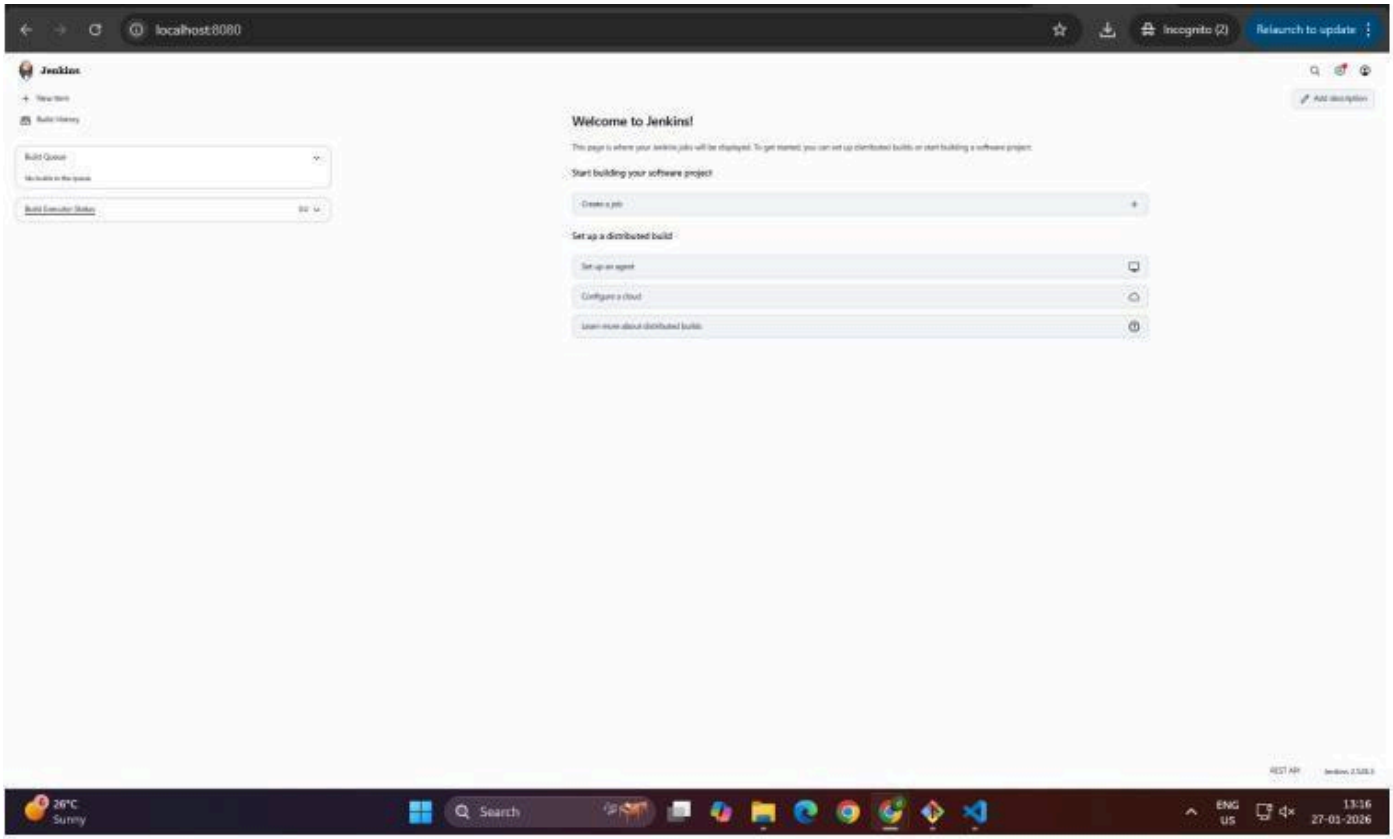
**Name : Kunal Motwani** **Batch : T21** **Roll No : 72**



**Name : Kunal Motwani** **Batch : T21** **Roll No : 72**






## Conclusion :

Thus, Continuous Integration was successfully understood and implemented by installing and configuring Jenkins along with build tools such as Maven, Ant, or Gradle. A build job was created and executed, demonstrating how code can be automatically built and tested. This process highlights the importance of automation in improving software quality, reducing errors, and enhancing team collaboration during development.